

Class 17: Evaluation and Decision-Making

Data-driven Systems Engineering

Agenda

- Evaluation must match the task, not the other way around.
- Why accuracy can fail badly on rare-event problems.
- From a prediction to a decision: what interfaces must communicate.
- Thresholds, uncertainty, and operational cost.
- Where trust in a model interface breaks down.

Learning goals

- Choose evaluation metrics that reflect the real decision, not a generic leaderboard score.
- Explain why accuracy is misleading under class imbalance.
- Distinguish a prediction from a decision, and identify what a decision-support interface must expose.
- Recognize the practical failure modes that erode trust in a served model.

Course Map and Running Cases



Today in the course

Focus: Evaluating models against the real task, and turning scores into trustworthy decisions.

Bridge: Class 16 gave us tracked, comparable model runs. This class asks whether the winning run is actually good for the decision it supports, and how that decision reaches a human or a downstream system.

Fraud case

In the fraud case, the key question is how to score rare events with metrics and thresholds that reflect the cost of a missed fraud versus a false alarm.

Pasteurization case

In the pasteurization case, a dashboard must communicate sensor state and model output together so an operator can act, not just observe a number.

Course thread: The course thread now shifts from tracked experimentation to decision quality and operational trust

Evaluation must match the task

- Imbalanced classification needs precision, recall, PR-AUC, and threshold analysis, not accuracy alone.
- Regression needs error-magnitude metrics such as MAE or RMSE, chosen for the unit that matters operationally.
- Calibrated probabilities matter whenever a downstream action depends on confidence, not just the predicted class.
- Operational cost matters as much as any single numerical score.

Fraud dataset reality: extreme class imbalance



From 'Credit_Card_Fraud_Detection_Solution.ipynb': the first modeling challenge is not algorithm choice but the rarity of the positive class.

Repository example: fit, predict, and inspect errors

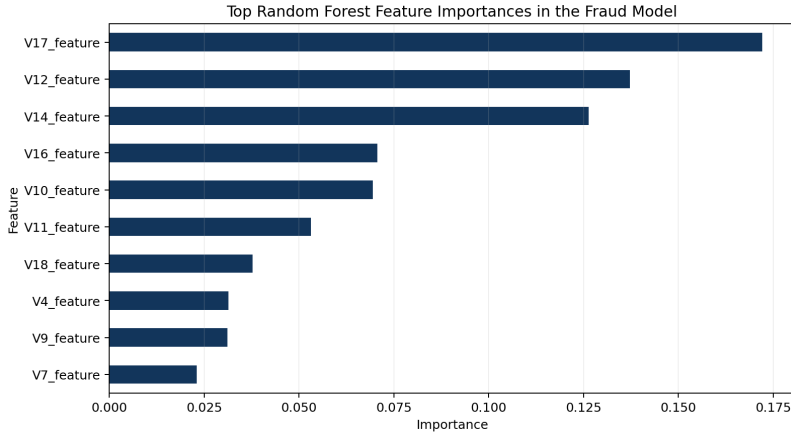
```
y_pred = best_model.predict(X_test_scaled)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

- From 'Credit_Card_Fraud_Detection_Solution.ipynb': the confusion matrix exposes missed fraud cases directly, which a single accuracy number cannot.
- The classification report opens the door to threshold and cost-sensitive evaluation.

Model interpretation: top feature importances



Generated from the saved 'streamlit/model.pkl': tree-based models can highlight influential variables, but importance is still not the same as causality.

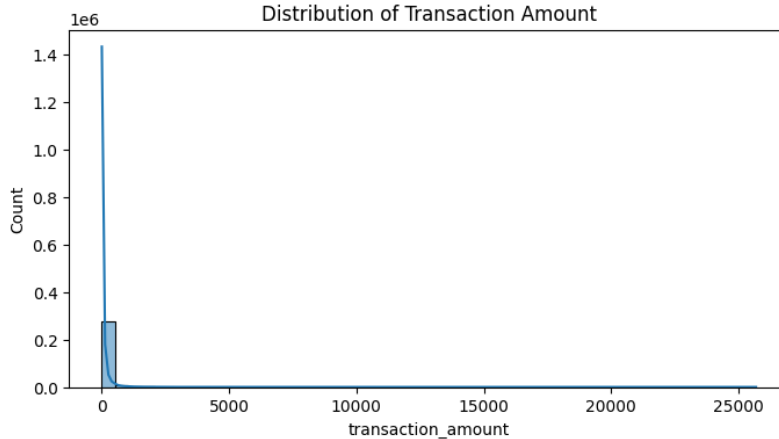
Why accuracy can fail badly

Fraud example

If only 0.2% of transactions are fraudulent, a model that predicts “legitimate” every time may look highly accurate while being operationally useless.

- This is why threshold, recall, and precision are part of the engineering conversation, not an afterthought.

Distribution shape interacts with evaluation



The transaction amount distribution is highly skewed, a reminder that evaluation, scaling, and threshold choice all interact with each other.

A model creates value only through an interface

- A trained model that nobody can call or interpret has limited practical value, however good its offline metrics look.
- An interface makes predictions available to users, services, and operational workflows.
- The interface also shapes trust: input checks, response format, and clarity all matter as much as the metric that selected the model.

Prediction is not the final decision

- A fraud score suggests escalation, not guilt.
- A process alert suggests investigation, not automatic shutdown.
- Interfaces should communicate uncertainty, threshold logic, and next actions whenever possible.

Engineering implication

The threshold that turns a probability into a decision is itself a design choice, not a byproduct of the model.

Repository example: threshold-driven decision support

```
if st.button("Predict Fraud Risk"):
    prediction = model.predict(scaled)[0]
    probability = model.predict_proba(scaled)[0][1]

    if prediction == 1:
        st.error(f"Fraud Probability: {probability:.2%}")
    else:
        st.success(f"Fraud Probability: {probability:.2%}")
```

- From 'streamlit/pages/App_Credit_Card.py': the interface exposes both the decision and the probability behind it.
- A human user needs labels, context, and actionable feedback, not raw class labels or JSON.

What a better dashboard should include

- A visible explanation of what the model does and does not do.
- Controls that reflect meaningful business inputs, not arbitrary technical internals.
- Results that support action: alert, score, trend, or explanation.
- Consistent links to data provenance, model version, or monitoring signals.

Where trust breaks down

- The interface hides uncertainty behind a single confident-looking number.
- The inputs shown to users do not match the assumptions the model was trained under.
- The threshold was chosen once, informally, and never revisited as the data changed.
- The dashboard makes a demo model look authoritative when it has not been validated for that purpose.

Repo activity

- 'Credit_Card_Fraud_Detection_Solution.ipynb' supplies the evaluation workflow this class is grounded in.
- 'streamlit/pages/App_Credit_Card.py' turns the same model into a threshold-driven, human-facing decision.
- Both artifacts share one model and one scaler, so evaluation and decision-making must stay consistent with each other.

Question

The fraud model has high recall but noticeably lower precision at the deployed threshold. Should we ship it as-is?

- This forces students to separate metric quality from decision quality and operational cost.
- It also motivates the online adaptation and serving discussions in Classes 18 and 20.

Papers and materials

scikit-learn model evaluation guide; Domingos (2012), A Few Useful Things to Know About Machine Learning; Chip Huyen, Designing Machine Learning Systems; Streamlit documentation.

From This Class to the Next

What stays with us

- Metrics are decision tools, chosen to reflect the real task, not decorative scoreboards.
- A prediction becomes a decision only once thresholds, uncertainty, and interface design are made explicit.
- Trust in a model interface is earned through validation, clarity, and honest communication of limits.

Project use

Project teams should now be able to defend both the metric they optimized and the threshold or interface behavior built on top of it.

Next connection

Class 18 asks what happens when the data underneath a validated model starts to change: online learning and concept drift.

Course thread: A model is only as trustworthy as its evaluation today and its ability to keep being evaluated as the world changes.